COMS30035, Machine learning: Introduction to Neural Networks

James Cussens

School of Computer Science
University of Bristol

7th September 2026

Acknowledgement

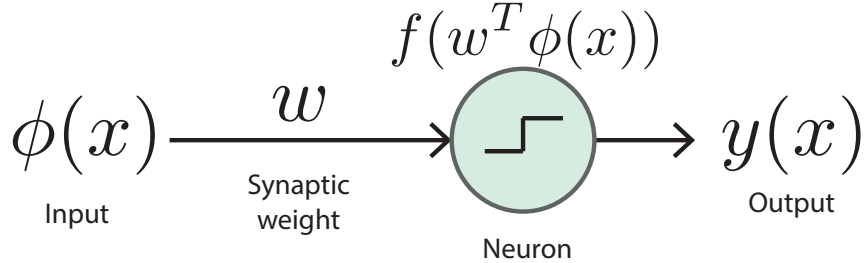
- These slides are adapted from ones originally created by Rui Ponte Costa and later edited by Edwin Simpson, James Cussens and Xiyue Zhang.

Agenda

- Perceptron
- Neural networks (multi-layer perceptron)
  - Architecture
  - The backpropagation algorithm
  - Gradient descent

Perceptron – a simplified neural network

- The very beginning of neural network models in ML!
- Directly inspired by how neurons process information:

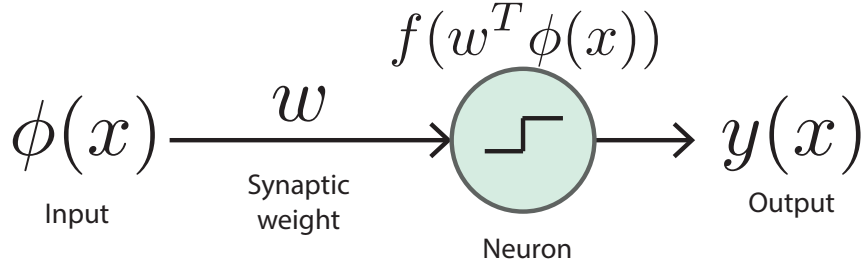


- It is an example of a linear discriminant model given by $y(\boldsymbol{x}) = f(\boldsymbol{w}^T\phi(\boldsymbol{x}))$

  with a nonlinear *activation function* $f(a) = \begin{cases} +1, & a \geq 0 \\ -1, & a < 0 \end{cases}$
- Here the target $t = \{+1, -1\}$.
- We aim to minimise the following error $-\sum_{n\in\mathcal{M}} \boldsymbol{w}^T\phi_n t_n$, where $\mathcal{M}$ denotes the misclassified datapoints and $\phi_n = \phi(\boldsymbol{x}_n)$.

Perceptron – a simplified neural network

- The very beginning of neural network models in ML!
- Directly inspired by how neurons process information:



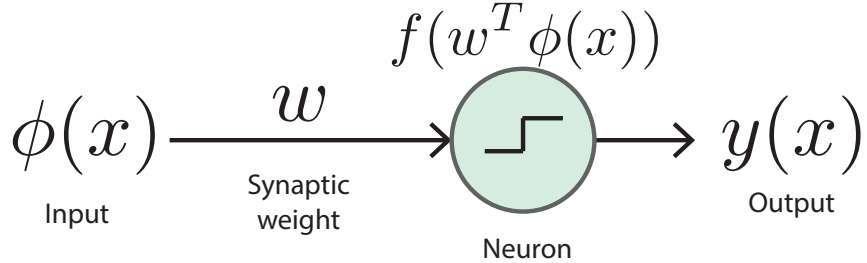
- It is an example of a linear discriminant model given by $y(\boldsymbol{x}) = f(\boldsymbol{w}^T\phi(\boldsymbol{x}))$

  with a nonlinear *activation function* $f(a) = \begin{cases} +1, & a \geq 0 \\ -1, & a < 0 \end{cases}$

- Here the target $t = \{+1, -1\}$.
- We aim to minimise the following error $-\sum_{n\in\mathcal{M}} \boldsymbol{w}^T\phi_n t_n$, where $\mathcal{M}$ denotes the misclassified datapoints and $\phi_n = \phi(\boldsymbol{x}_n)$.

Perceptron – a simplified neural network

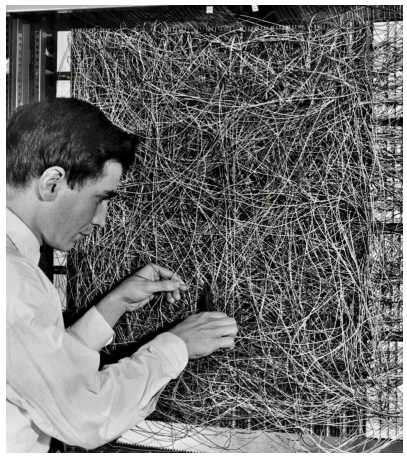
- The very beginning of neural network models in ML!
- Directly inspired by how neurons process information:



- It is an example of a linear discriminant model given by $y(\boldsymbol{x}) = f(\boldsymbol{w}^T\phi(\boldsymbol{x}))$

  with a nonlinear *activation function* $f(a) = \begin{cases} +1, & a \geq 0 \\ -1, & a < 0 \end{cases}$
- Here the target $t = \{+1, -1\}$.
- We aim to minimise the following error $-\sum_{n\in\mathcal{M}} \boldsymbol{w}^T\phi_n t_n$, where $\mathcal{M}$ denotes the misclassified datapoints and $\phi_n = \phi(\boldsymbol{x}_n)$.

Perceptron – a simplified neural network



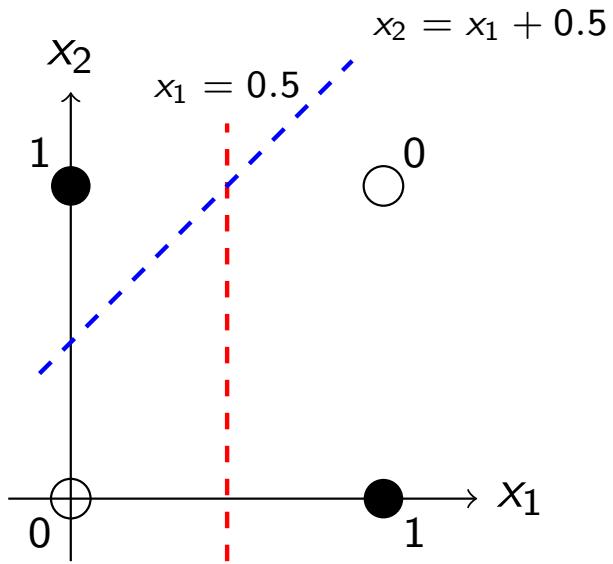
The Perceptron of Rosenblatt (1962)

Perceptrons started the journey to the current *deep learning* revolution!
Frank Rosenblatt used IBM and special-purpose hardware for a parallel implementation of perceptron learning.
Marvin Minksy, showed that such models could only learn *linearly separable problems*.
For example, the XOR problem could not be solved by the perceptron model.

source: Bishop p193.

XOR problem

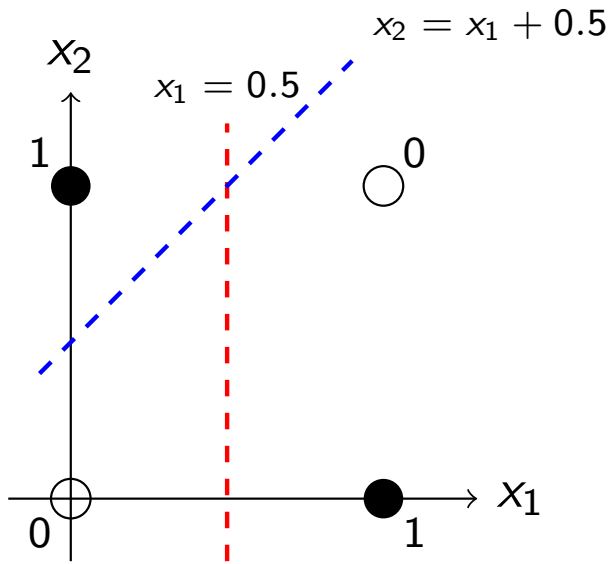
- Label 1: Points $(1, 0)$ and $(0, 1)$
- Label 0: Points $(0, 0)$ and $(1, 1)$



No single straight line separates the positive and negative points. This limitation can be solved by a multi-layer perceptron (MLP).

XOR problem

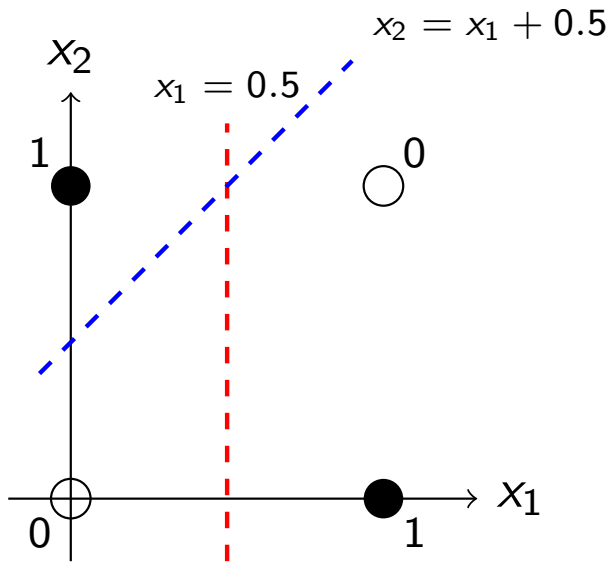
- Label 1: Points $(1, 0)$ and $(0, 1)$
- Label 0: Points $(0, 0)$ and $(1, 1)$



No single straight line separates the positive and negative points.
This limitation can be solved by a multi-layer perceptron (MLP).

XOR problem

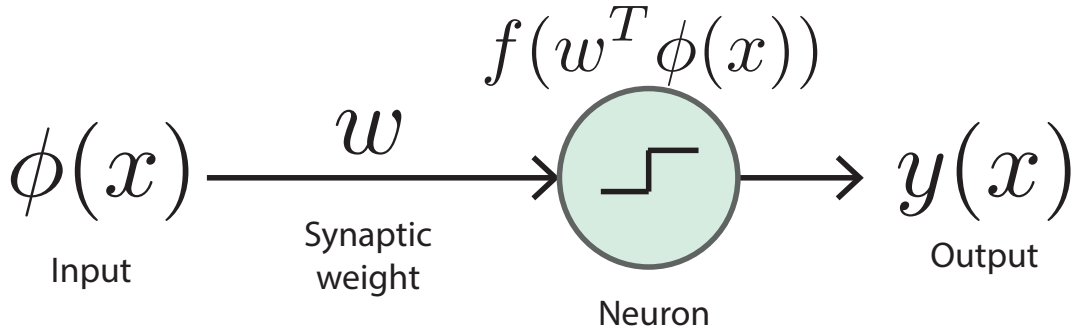
- Label 1: Points $(1, 0)$ and $(0, 1)$
- Label 0: Points $(0, 0)$ and $(1, 1)$



No single straight line separates the positive and negative points.
This limitation can be solved by a multi-layer perceptron (MLP).

Neural networks

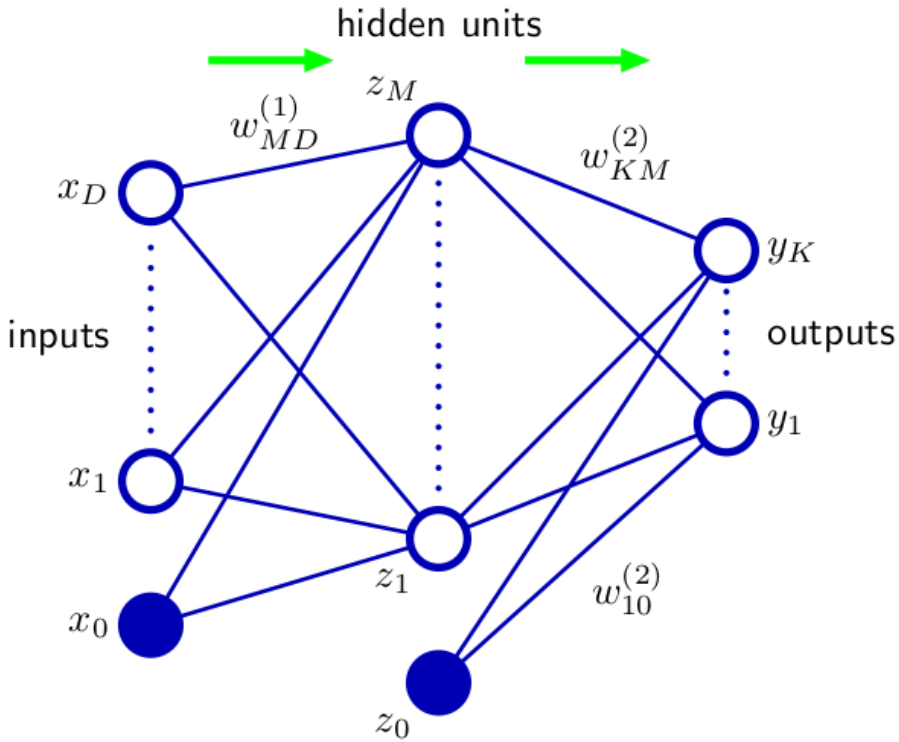
From a single layer perceptron:



Neural networks

To a two-layer neural network (from Bishop):

Figure 5.1 Network diagram for the two-layer neural network corresponding to (5.7). The input, hidden, and output variables are represented by nodes, and the weight parameters are represented by links between the nodes, in which the bias parameters are denoted by links coming from additional input and hidden variables x_0 and z_0. Arrows denote the direction of information flow through the network during forward propagation.



If we use a sigmoidal output unit *activation function* then the overall network function would be:

$$y_k(\boldsymbol{x}, \boldsymbol{w}) = \sigma\left(\sum_{j=1}^{M} w_{kj}^{(2)} h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) + w_{k0}^{(2)}\right)$$

where h is some differentiable, nonlinear activation function.

Neural networks

- Neural networks are at heart composite functions of linear-nonlinear functions.
- **Deep learning** refers to neural networks with more than 1 hidden layer
- They can be applied in any form of learning, but we will focus on supervised learning and classification in particular
- Neural network recipe [1]:
  - Define architecture (i.e. how many hidden layers and neurons) [2]
  - Define cost function (e.g. mean squared error)
  - Optimise network using backprop:
    1. Forward pass – calculate activations; generate y_k
    2. Calculate error/cost function
    3. Backward pass – use backprop to compute error gradient
    4. Use gradient to improve parameters

[1] Here we focus on simple feedforward nets but the recipe is the same for any neural network.

[2] Note that this makes them parametric models.

Neural networks

- Neural networks are at heart composite functions of linear-nonlinear functions.
- **Deep learning** refers to neural networks with more than 1 hidden layer
- They can be applied in any form of learning, but we will focus on supervised learning and classification in particular
- Neural network recipe [1]:
  - Define architecture (i.e. how many hidden layers and neurons) [2]
  - Define cost function (e.g. mean squared error)
  - Optimise network using backprop:
    1. Forward pass – calculate activations; generate y_k
    2. Calculate error/cost function
    3. Backward pass – use backprop to compute error gradient
    4. Use gradient to improve parameters

[1] Here we focus on simple feedforward nets but the recipe is the same for any neural network.

[2] Note that this makes them parametric models.

Neural networks – forward pass step-by-step

Assume just one hidden layer and σ for the output activation function...

1. Calculate activations of the hidden layer: $a_j = \sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}$ [linear]
2. Pass it through a nonlinear function: $z_j = \sigma(a_j)$ [nonlinear]
3. Calculate activations of the output layer: $a_k = \sum_{j=1}^{M} w_{kj}^{(2)} z_j + w_{k0}^{(2)}$ [linear]
4. Compute predictions using a sigmoid: $y_k = \sigma(a_k)$ [nonlinear]
5. All together: $y_k = \sigma\left(\sum_{j=1}^{M} w_{kj}^{(2)} h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) + w_{k0}^{(2)}\right)$

Neural networks – forward pass step-by-step

Assume just one hidden layer and σ for the output activation function...

1. Calculate activations of the hidden layer: $a_j = \sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}$ [linear]
2. Pass it through a nonlinear function: $z_j = \sigma(a_j)$ [nonlinear]
3. Calculate activations of the output layer: $a_k = \sum_{j=1}^{M} w_{kj}^{(2)} z_j + w_{k0}^{(2)}$ [linear]
4. Compute predictions using a sigmoid: $y_k = \sigma(a_k)$ [nonlinear]
5. All together: $y_k = \sigma\left(\sum_{j=1}^{M} w_{kj}^{(2)} h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) + w_{k0}^{(2)}\right)$

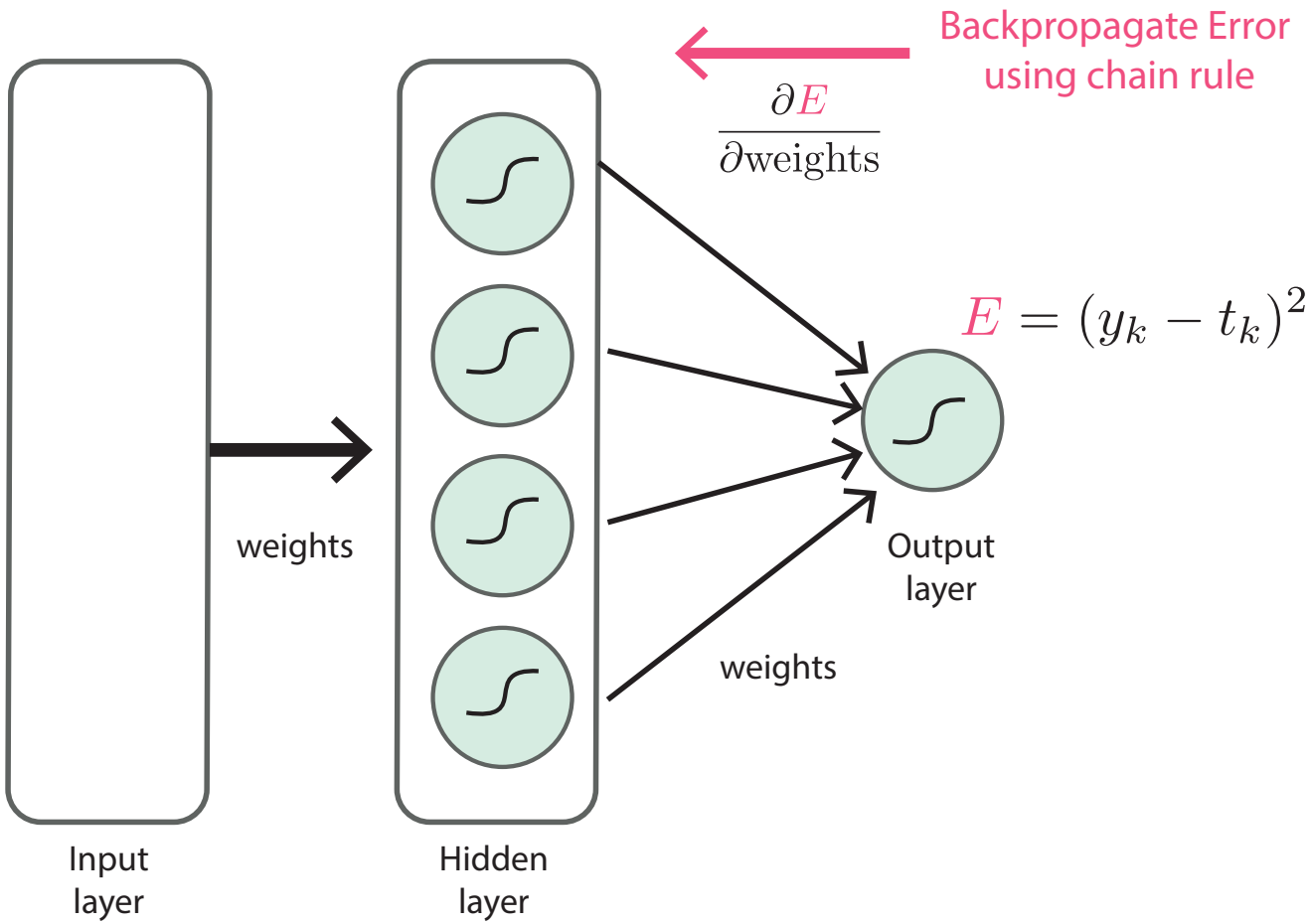
Neural networks – forward pass step-by-step

Assume just one hidden layer and σ for the output activation function...

1. Calculate activations of the hidden layer: $a_j = \sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}$ [linear]
2. Pass it through a nonlinear function: $z_j = \sigma(a_j)$ [nonlinear]
3. Calculate activations of the output layer: $a_k = \sum_{j=1}^{M} w_{kj}^{(2)} z_j + w_{k0}^{(2)}$ [linear]
4. Compute predictions using a sigmoid: $y_k = \sigma(a_k)$ [nonlinear]
5. All together: $y_k = \sigma\left(\sum_{j=1}^{M} w_{kj}^{(2)} h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) + w_{k0}^{(2)}\right)$

Neural networks – backward pass

We now need to optimise our weights, and as before we use derivatives to find a solution. Effectively backpropagating the output error signal across the network – backpropagation algorithm.



This is a special case where the output layer has a single node. In the general case we could have e.g. $E = \sum_k (y_k - t_k)^2$.

Total error as a sum of datapoint errors

- Typically the error for an entire dataset breaks down into a sum of errors for each individual datapoint:

$$E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w})$$

- This means that the error gradient for the entire dataset with respect to some weight w_{ji} will just be the sum of error gradients for each datapoint:

$$\frac{\partial E}{\partial w_{ji}} = \sum_{n=1}^{N} \frac{\partial E_n}{\partial w_{ji}}$$

- So we focus attention on computing the $\frac{\partial E_n}{\partial w_{ji}}$ values.

Computing partial derivatives 1

- Although all values in the network depend on the datapoint n we're focusing on, we will (like Bishop) omit the subscript n from network values to reduce clutter.
- Consider an arbitrary unit j (aka 'node') in an neural network.
- We will associate the *bias* for a unit with a dummy input fixed to 1.
- The unit computes a value z_j by first computing a_j, a weighted sum of its inputs (from the previous layer), and then sending a_j to some nonlinear *activation function* h:

$$a_j = \sum_i w_{ji} z_i \tag{1}$$

$$z_j = h(a_j) \tag{2}$$

$$\tag{3}$$

- Since E_n (the error for the nth datapoint) only depends on weight w_{ji} via a_j we can use the chain rule to write:

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial w_{ji}}$$

Computing partial derivatives 2

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial w_{ji}}$$

- Since $a_j = \sum_i w_{ji} z_i$, $\frac{\partial a_j}{\partial w_{ji}}$ is easy to compute:

$$\frac{\partial a_j}{\partial w_{ji}} = z_i$$

As for $\frac{\partial E_n}{\partial a_j}$ let's just introduce some notation for now: $\delta_j \equiv \frac{\partial E_n}{\partial a_j}$

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i \tag{4}$$

Computing partial derivatives 3

- We compute the $\delta_j \equiv \frac{\partial E_n}{\partial a_j}$ *backwards*, starting with the output units.
- For example, if the error (for a single datapoint) is $E_n = \frac{1}{2}\sum_k (y_k - t_k)^2$ where $y_k = h(a_k)$ then:

$$\delta_k \equiv \frac{\partial E_n}{\partial a_k} = \frac{\partial E_n}{\partial y_k}\frac{\partial y_k}{\partial a_k} = (y_k - t_k)\, h'(a_k) \tag{5}$$

- For the hidden units we use (a multivariable version of) the chain rule:

$$\delta_j \equiv \frac{\partial E_n}{\partial a_j} = \sum_k \frac{\partial E_n}{\partial a_k}\frac{\partial a_k}{\partial a_j}$$

"where the sum runs over all units k to which unit j sends connections" (Bishop). "we are making use of the fact that variations in a_j give rise to variations in the error function only through variations in the variables a_k." (Bishop).

Computing partial derivatives 4

$$\delta_j \equiv \frac{\partial E_n}{\partial a_j} = \sum_k \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j}$$

can be reformulated to give the *backpropagation formula*:

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k \tag{6}$$

Error Backpropagation

As given in Bishop:

1. Apply an input vector $\mathbf{x}_n$ to the network and forward propagate through the network using (1) and (2) to find the activations of all the hidden and output units.
2. Evaluate the δ_k for all the outputs units using e.g. (5).
3. Backpropagate the δ's using (6) to obtain δ_j for each hidden unit in the network.
4. Use (4) to evaluate the required derivatives.

- A key point is that computing a δ_j value helps us compute **several** other $\delta_{j'}$ values.

Backpropagation in practice

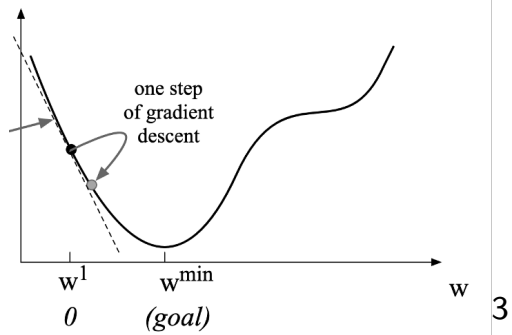
- It is crucial to organise the computation of all these partial derivatives (i.e. the gradient $\partial E/\partial \mathbf{w}$) efficiently.
- The quantities required will be organised in tensors (generalisation of matrices beyond 2 dimensions).
- The `PyTorch` approach is called *Autograd*. There's lots of useful info/explanation of how it works on the PyTorch site.

PyTorch

> *PyTorch's Autograd feature is part of what make PyTorch flexible and fast for building machine learning projects. It allows for the rapid and easy computation of multiple partial derivatives (also referred to as gradients) over a complex computation. This operation is central to backpropagation-based neural network learning.* *The Fundamentals of Autograd*

Neural networks – gradient descent [4]

In many ML methods it is common to iteratively update the parameters by descending the gradient.



[3]

In our neural network (one hidden layer) this means to update the weights using:

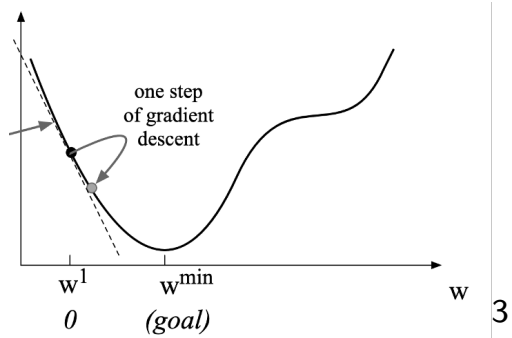
- Output: $w_{kj}^{new} = w_{kj}^{old} - \Delta w_{kj}$, where $\Delta w_{kj} = h'(a_k)(y_k - t_k)z_j$
- Hidden $w_{ji}^{new} = w_{ji}^{old} - \Delta w_{ji}$, where $\Delta w_{ji} = h'(a_j)\sum_k w_{kj}(y_k - t_k)h'(a_k)x_i$
- $w_{kj}^{new} = w_{kj}^{old} - \eta\,\Delta w_{kj}$, where η denotes the learning rate.
- This is often done in mini-batches – using a small number of samples to compute Δw.

[3] Figure from https://mc.ai/an-introduction-to-gradient-descent-2/ (link no longer working)

[4] Its called descent because we are minimising the cost function, so descending on the function landscape, which can be quite hilly!

Neural networks – gradient descent [4]

In many ML methods it is common to iteratively update the parameters by descending the gradient.



[3]

In our neural network (one hidden layer) this means to update the weights using:

- Output: $w_{kj}^{new} = w_{kj}^{old} - \Delta w_{kj}$, where $\Delta w_{kj} = h'(a_k)(y_k - t_k)z_j$
- Hidden $w_{ji}^{new} = w_{ji}^{old} - \Delta w_{ji}$, where $\Delta w_{ji} = h'(a_j)\sum_k w_{kj}(y_k - t_k)h'(a_k)x_i$
- $w_{kj}^{new} = w_{kj}^{old} - \eta\,\Delta w_{kj}$, where η denotes the learning rate.
- This is often done in mini-batches – using a small number of samples to compute Δw.

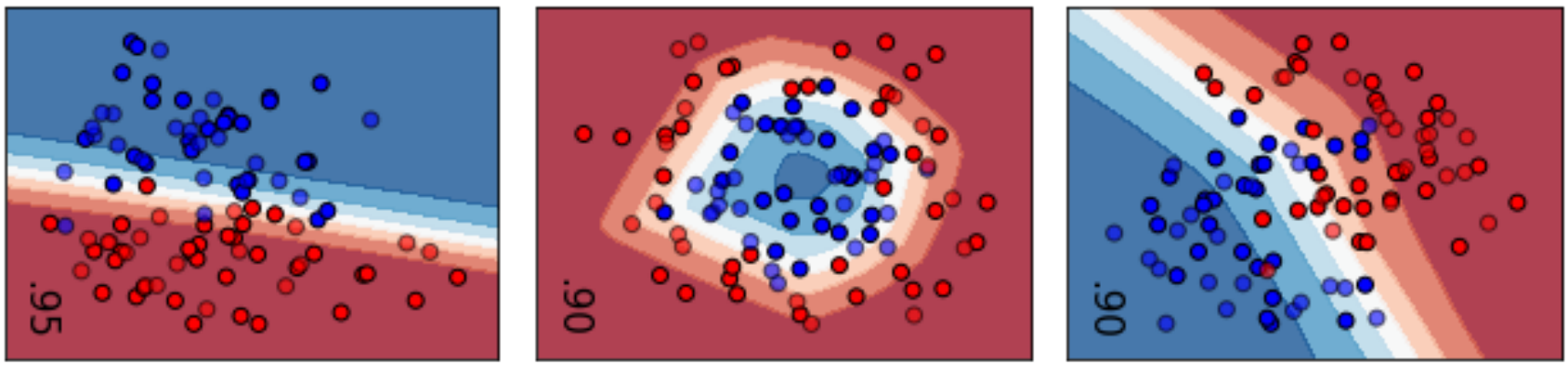
[3] Figure from https://mc.ai/an-introduction-to-gradient-descent-2/ (link no longer working)

[4] Its called descent because we are minimising the cost function, so descending on the function landscape, which can be quite hilly!

Neural networks

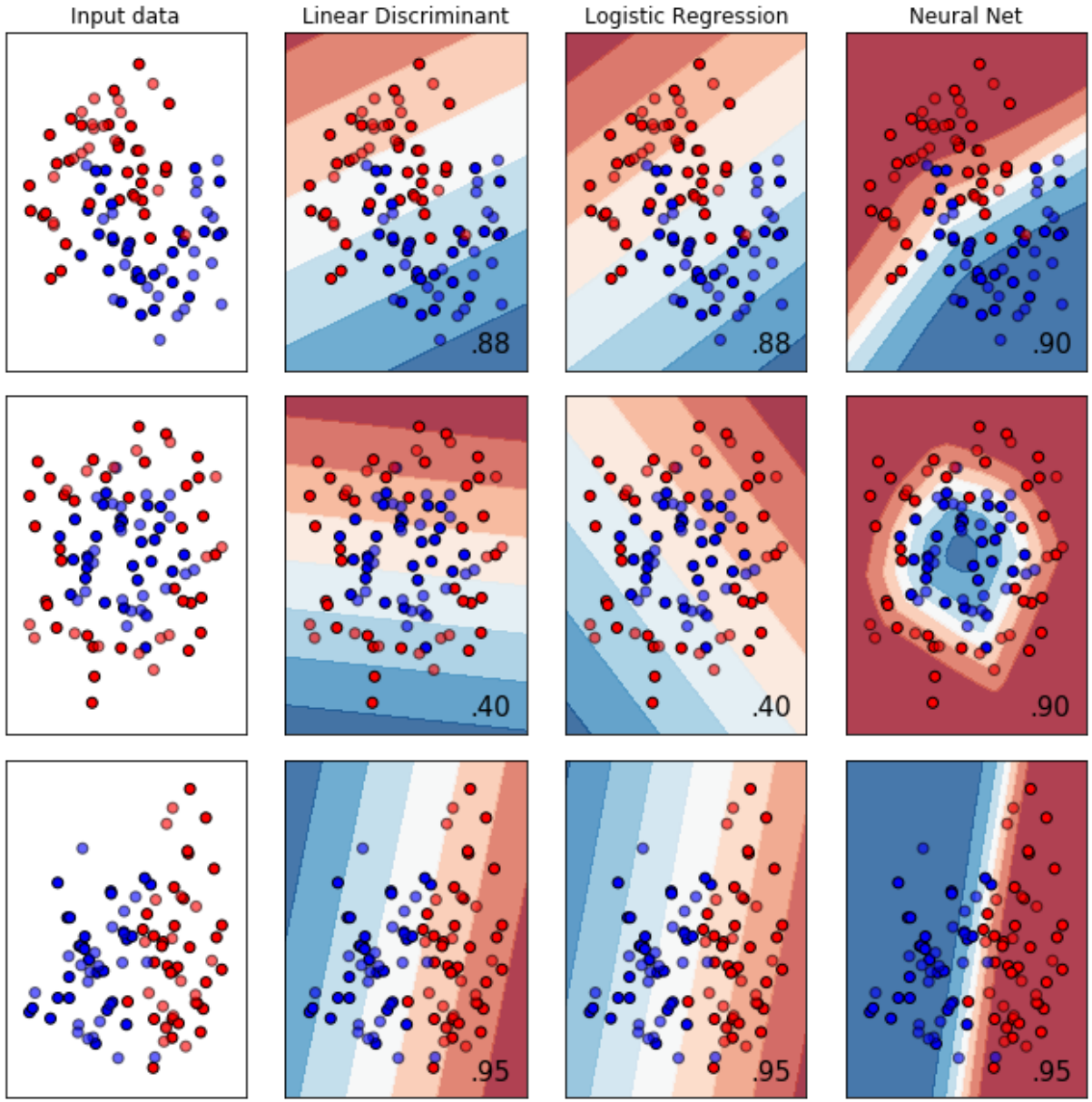
Example

Using sklearn we fitted a MLP classifier to the data:

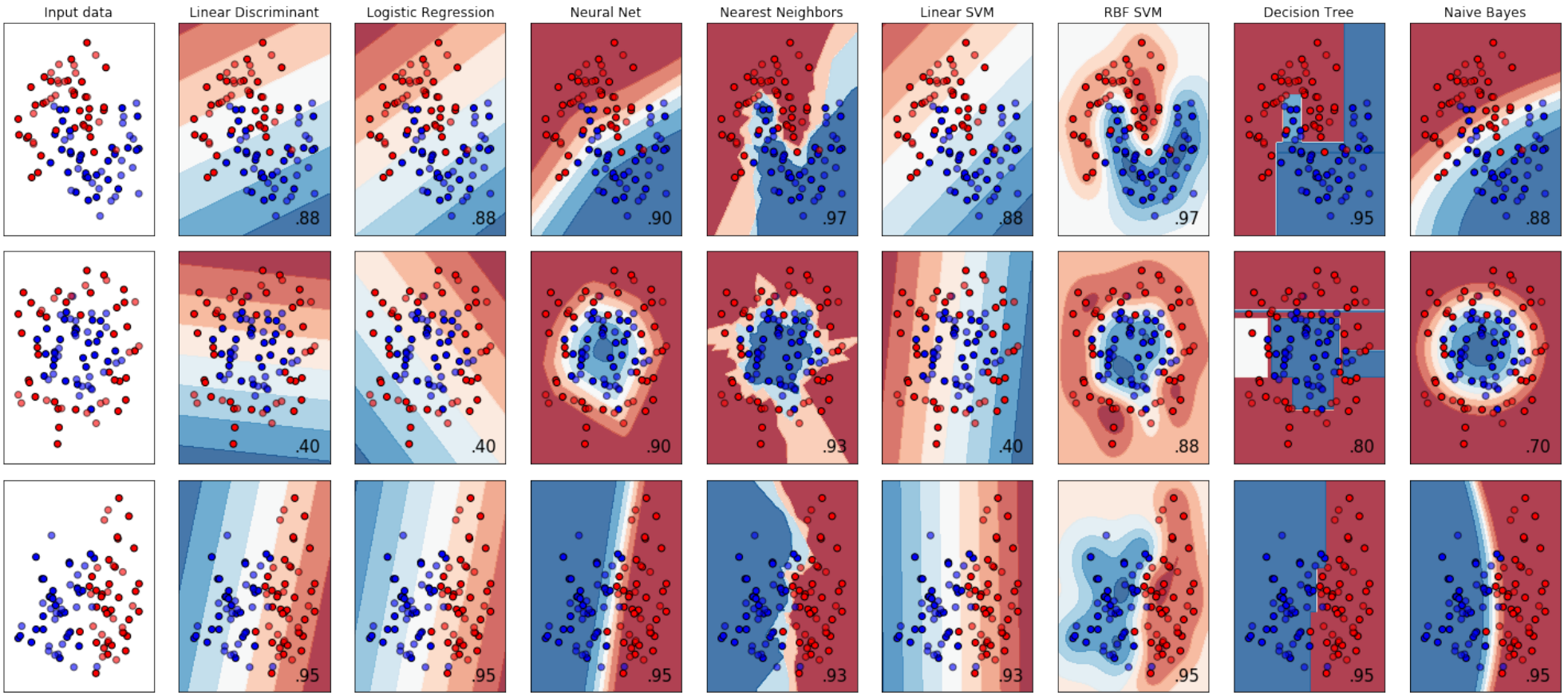


An MLP with one hidden layer can perform well in nonlinear classification problems.
However, because MLPs are highly flexible they can easily *overfit*.
Solutions: *early stopping* (stop when test performance starts decreasing) and *regularisation* methods such as *dropout* (randomly turn off units during training).

Classification methods – overall comparison

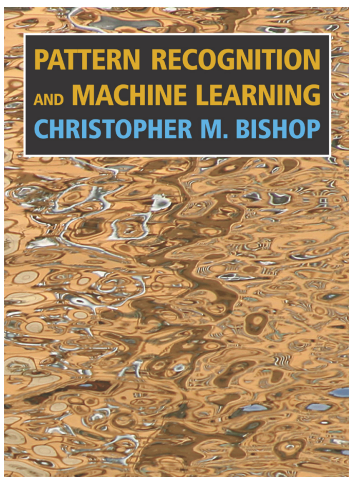


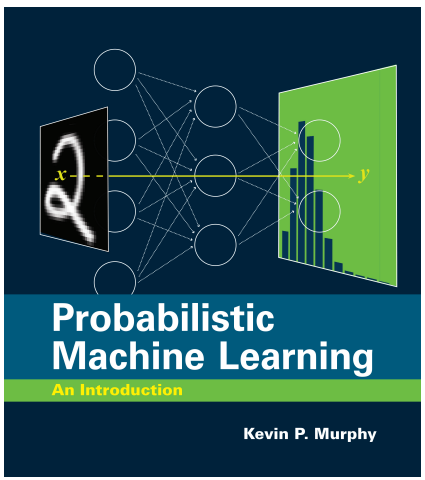
Classification methods – overall comparison [we don't cover all these methods]



Reading

- Bishop §5.1 except §5.1.1
- Bishop §5.2 except §5.2.2
- Bishop §5.3 up to §5.3.2
- Murphy §13.1-§13.2
- Murphy's description of backpropagation in §13.3 is interesting since it is closer to how it is actually implemented, but more complex than Bishop's presentation.





Exercises and lab/practical

- Bishop Exercise 4.12
- Bishop Exercise 5.6 (Tip: Use the result you derived in Exercise 4.12 to help you.)
- Lab/Practical:
  - Week 2: Neural Networks